

■ TECHNICAL DOCUMENTATION v1.0

ShareAnywhere

P2P File Share & Chat

WebRTC Direct · Zero Upload · Railway Ready

■ P2P Chat

■ QR Scan

■ Accept/Reject

■ Zero Server Up

MODULE

ShareAnywhere v1.0

STACK

Python · Flask · WebRTC

DEPLOY

Railway.app Ready

LICENSE

Proprietary

DEVELOPED BY

Mr. D.K. Chaudhary

@THEDARKGEEKDC · Primary Teacher & Cyber Security Enthusiast

Published: 10 July 2026



Table of Contents

1.	Project Overview	3
2.	Architecture & Flow	4
3.	Features	5
4.	Page-by-Page Breakdown	6
5.	Backend API Reference	8
6.	WebRTC & P2P Engine	9
7.	Installation & Deployment	10
8.	Security & Privacy	11
9.	Bug Fixes & Changelog	12
10.	Contact & Licensing	13

1

Project Overview

What is ShareAnywhere?

ShareAnywhere is a **real-time, peer-to-peer file sharing and chat application** built with Python (Flask + SocketIO) on the backend and pure HTML/CSS/JavaScript on the frontend. It enables users to transfer files and exchange messages **directly between browsers** using WebRTC DataChannels — without uploading any file to the server.

■ Files are NEVER stored on the server. The server only handles WebRTC signaling (offer/answer/ICE). All file data travels directly between browsers.

Key Highlights

Feature	Description
■ Ultra Fast	Same-WiFi transfers use LAN path (~500 MB/s). Internet uses WebRTC STUN P2P.
■ Accept/Reject	Sender receives a popup modal before any connection is established.
■ P2P Chat	Real-time messaging over WebRTC DataChannel — no server relay.
■ QR Scan	Camera-based QR scanner with html5-qrcode. Auto-fills code & connects.
■ Multi-File	Send multiple files at once. Receiver gets a ZIP if multiple files.
■ Multi-Receiver	Multiple receivers can join the same room simultaneously.
■ Auto Network	Server auto-detects WiFi vs Internet — no manual selection needed.
■ Railway Ready	Includes Procfile, railway.toml, .python-version for 1-click deploy.

Technology Stack

Feature	Description
Python 3.11	Core backend runtime
Flask 3.x	Lightweight WSGI web framework
Flask-SocketIO 5.x	WebSocket signaling server
eventlet	Async worker (monkey-patched before all imports)
qrcode + Pillow	Server-side QR code image generation
Socket.IO 4.7 (JS)	Client-side WebSocket connection
WebRTC (native)	Browser P2P DataChannel — no library needed
JSZip 3.x	Browser-side ZIP creation for multi-file download

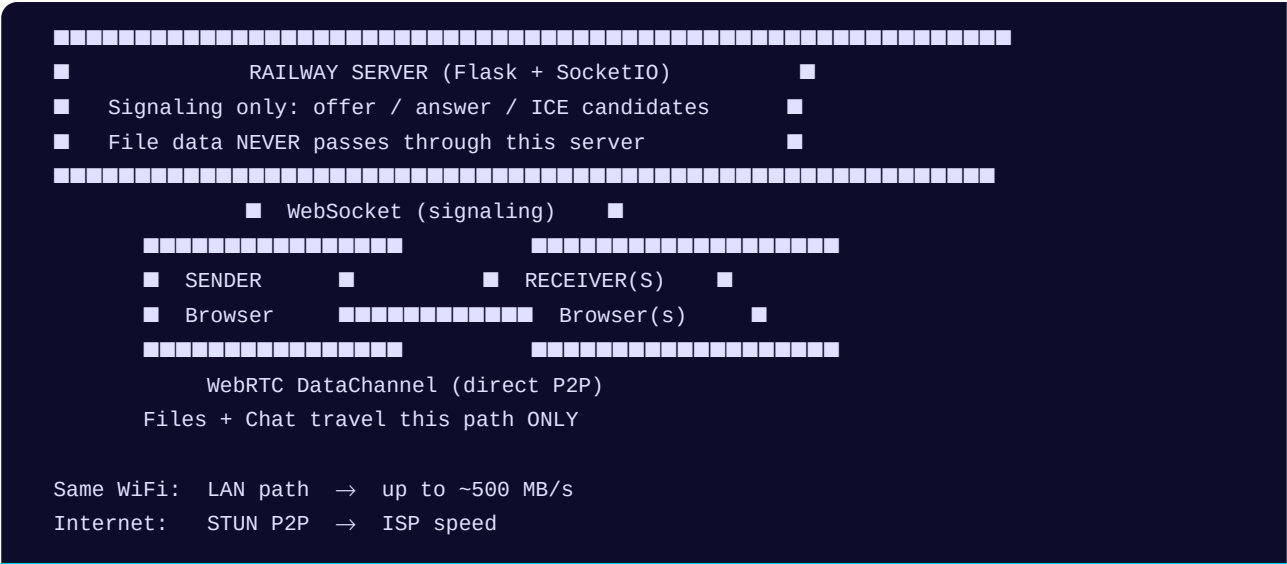
html5-qrcode 2.x	Camera QR scanner in browser
Gunicorn + eventlet	Production WSGI server for Railway

2

Architecture & Data Flow

System Architecture

ShareAnywhere follows a **signaling-only server** model. The server never touches file data — it only acts as a message relay to establish WebRTC peer connections.



Connection Flow (Step by Step)

Step	Actor	Event	Description
1	Sender	<i>create_room</i>	Server creates a 4-digit room code
2	Receiver	<i>join_room_request</i>	Receiver submits code → request queued
3	Server	<i>incoming_request</i>	Server relays request to sender
4	Sender	<i>request_decision</i>	Sender clicks Accept / Reject in modal
5	Server	<i>request_accepted</i>	Server adds receiver to SocketIO room
6	Sender	<i>webrtc_offer</i> →	SDP offer sent via socket to receiver
7	Receiver	<i>webrtc_answer</i> →	SDP answer returned to sender
8	Both	<i>webrtc_ice</i> ↔	ICE candidates exchanged (STUN)
9	Both	<i>DataChannel open</i>	Direct P2P established — files & chat flow
10	Receiver	<i>all_done</i> → <i>download</i>	Browser assembles file/ZIP and prompts save

3

Features Deep Dive

■ Accept / Reject System

When a receiver submits a room code, a pending request is created on the server. The sender's UI shows a modal popup: 'Incoming Connection — Accept / Reject'. Only after acceptance does the WebRTC handshake begin. Rejected receivers see an error message. This prevents unauthorized connections entirely.

■ P2P Chat

Chat messages travel through the same WebRTC DataChannel as file data, using a simple JSON envelope: `{t:'chat', msg:'...'}`. Both sender and receiver can type messages at any time. URLs are auto-detected and rendered with Open + Copy buttons. The chat textarea auto-resizes and supports Shift+Enter for newlines.

■ QR Scan / Enter Code Toggle

The receive page has a single smart toggle: 'Enter Code' and 'Scan QR'. Switching to Scan opens the device camera using `html5-qrcode` library. On a successful scan, the code is extracted from the URL or used directly, the scanner closes, and `joinRoom()` is called automatically.

■ File Sharing Engine

Files are read in 4 MB blocks from disk and sent as 256 KB WebRTC chunks with backpressure control (`bufferedAmountLowThreshold`). A JSON control message precedes each binary chunk. Multiple files are tracked per-index. On completion, a single file downloads directly; multiple files are zipped browser-side with JSZip (compression level 1).

■ Auto Network Detection

The `/api/network` endpoint checks if the request host is a private IP (RFC1918) or localhost — if yes, `mode='wifi'` (up to 5 GB). Otherwise `mode='inet'` (up to 500 MB). The home page shows a live chip: 'Local WiFi' or 'Internet'. Two QR codes are generated in WiFi mode (LAN + Internet URLs).

■ Multi-Receiver Support

Each receiver gets an independent `RTCPeerConnection` and `DataChannel`. The sender maintains a `peers{}` map keyed by socket ID. Files are sent to all peers via `sendFilesNow()`. Each receiver row shows live progress bar, percentage, and status.

Page-by-Page UI Breakdown

■ Home Page (pgHome)

- App logo + auto-detected network chip (WiFi/Internet)
- Two primary action buttons: ■ Share and ■ Receive
- Feature pills row: Multiple files · P2P Chat · QR Scan · Accept/Reject · Auto-deleted
- Footer with full developer credit on every page
- No login, no name entry — completely frictionless

■ Share Page (pgSend)

- Step 1 — Create Room: Optional file drop zone + Create Room button
- Step 2 — Active Room: 4-digit code display + Copy Code / Copy Link buttons
- Dual QR codes shown in WiFi mode (LAN QR + Internet QR)
- Connected users strip with avatar, name, status, progress bar, percentage
- Tab bar: ■ Chat tab and ■ Share Files tab
- Chat panel: auto-resize textarea, Shift+Enter newlines, URL auto-detect
- Files panel: second drop zone, file list, Send Files to All button
- Incoming request → overlay modal (Accept / Reject)

■ Receive Page (pgRecv)

- Smart toggle: ■■ Enter Code ↔ ■ Scan QR (single button, no separate pages)
- OTP-style 4 boxes — tap anywhere to open keyboard on mobile
- Paste support: onpaste with setTimeout ensures correct value
- QR scanner: html5-qrcode, auto-extracts code from URL or raw digits
- Step 2 — Waiting: spinner shown while awaiting sender's Accept/Reject
- Step 3 — Progress: per-file progress bars + overall bar + speed/ETA metrics
- Receiver chat panel mirrors sender chat (full duplex P2P)

■ Navbar (with back button + network dot) and Footer (@THEDARKGEEKDC credit) appear on every page — no exceptions.

5

Backend API Reference

HTTP Endpoints

Feature	Description
<code>GET /</code>	Serves index.html (frontend SPA)
<code>GET /api/localip</code>	Returns server's local IP and port as JSON
<code>GET /api/network</code>	Returns mode (wifi/inet), label, local_ip, port
<code>GET /qr?d=URL&s=N</code>	Generates PNG QR code. d=data URL, s=box size (4-20, default 9)

Socket.IO Events

Client → Server:

Feature	Description
<code>create_room</code>	Sender creates a room. Server returns <code>room_created {code}</code>
<code>join_room_request</code>	Receiver submits <code>{code}</code> . Server queues request, notifies sender
<code>request_decision</code>	<code>{request_id, decision:'accept' 'reject'}</code> . Sender approves/rejects
<code>webrtc_offer</code>	<code>{to:sid, offer:SDP}</code> . Relayed to target receiver
<code>webrtc_answer</code>	<code>{to:sid, answer:SDP}</code> . Relayed to target sender
<code>webrtc_ice</code>	<code>{to:sid, candidate:ICE}</code> . Relayed to target peer

Server → Client:

Feature	Description
<code>room_created</code>	<code>{code}</code> — sent to sender after room creation
<code>incoming_request</code>	<code>{request_id, receiver_sid}</code> — sent to sender on join attempt
<code>waiting_for_approval</code>	<code>{request_id}</code> — sent to receiver to show waiting UI
<code>request_accepted</code>	<code>{request_id, code, sender_sid}</code> — sent to receiver on accept
<code>request_rejected</code>	<code>{request_id}</code> — sent to receiver on reject
<code>request_canceled</code>	<code>{request_id}</code> — sent if sender disconnects during pending
<code>receiver_joined</code>	<code>{receiver_sid}</code> — sent to sender after accept
<code>webrtc_offer</code>	<code>{offer, from}</code> — relayed SDP offer to receiver
<code>webrtc_answer</code>	<code>{answer, from}</code> — relayed SDP answer to sender
<code>webrtc_ice</code>	<code>{candidate, from}</code> — relayed ICE candidate

receiver_left	{sid} — sent to sender when receiver disconnects
sender_left	{ } — broadcast to all receivers when sender disconnects
error	{msg} — error string for UI display

6

WebRTC & P2P Transfer Engine

ICE / STUN Configuration

```
const ICE_CFG = { iceServers: [
  { urls: 'stun:stun.l.google.com:19302' },
  { urls: 'stun:stun1.l.google.com:19302' },
  { urls: 'stun:stun.cloudflare.com:3478' },
]}

// 3 STUN servers for redundancy
// Works for ~85% of networks (symmetric NAT may fail)
// TURN server can be added as fallback if needed
```

Chunk Protocol (DataChannel messages)

Each file transfer uses a paired message pattern: a JSON control message followed immediately by a binary ArrayBuffer chunk.

```
// Step 1: Sender announces metadata
{t:"meta", totalSize:N, files:[{index, name, size, mime}]}

// Step 2: Per file – start signal
{t:"file_start", idx, name, size}

// Step 3: Repeated for each 256KB chunk
{t:"chunk", idx, offset, sz} ← JSON control
<ArrayBuffer data>          ← Binary chunk (next message)

// Step 4: File complete
{t:"file_done", idx}

// Step 5: All files done
{t:"all_done"}

// Chat message (any time, same channel)
{t:"chat", msg:"..."}
```

Backpressure Control

To prevent DataChannel buffer overflow on large files, the sender checks **dc.bufferedAmount** before each chunk. If it exceeds 8× chunk size, the sender awaits the **bufferedamountlow** event before continuing. The threshold is set once at channel creation (not reassigned in-loop).

```
dc.bufferedAmountLowThreshold = CHUNK * 2; // Set ONCE at creation

// In send loop:
if (dc.bufferedAmount > CHUNK * 8) {
  await new Promise(res => {
    const check = () => {
      if (dc.readyState !== 'open') { res(); return; }
      if (dc.bufferedAmount <= CHUNK * 2) { res(); return; }
      dc.onbufferedamountlow = () => { dc.onbufferedamountlow=null; res(); };
    };
    check();
  });
}
```

7

Installation & Deployment

Local Development

```
# 1. Install dependencies
pip install -r requirements.txt

# requirements.txt:
# flask>=3.0.0
# flask-socketio>=5.3.6
# eventlet>=0.35.2
# qrcode[pil]>=7.4.2
# Pillow>=10.0.0
# gunicorn>=21.2.0

# 2. Run server
python server.py

# 3. Open browser
# http://localhost:5683
```

Railway Deployment (Recommended)

```
# Procfile:
web: gunicorn --worker-class eventlet -w 1 --bind 0.0.0.0:$PORT server:app

# railway.toml:
[build]
builder = "NIXPACKS"

[deploy]
startCommand = "gunicorn --worker-class eventlet -w 1 --bind 0.0.0.0:$PORT server:app"
restartPolicyType = "ON_FAILURE"

# .python-version:
python-3.11.0
```

Deploy Steps

1. Push code to GitHub repository
2. Go to railway.app → New Project → Deploy from GitHub
3. Select your repository
4. Railway auto-detects Python + Procfile
5. Deploy! Your URL will be: <https://yourapp.up.railway.app>

■ Use only 1 Gunicorn worker (eventlet mode). Multiple workers will break WebSocket state — each worker has its own `rooms` dict.

8

Security & Privacy

Feature	Description
Zero server storage	File bytes NEVER reach the server. DataChannel is purely client-to-client.
Accept/Reject gate	No connection is established without explicit sender approval.
4-digit room codes	9000 possible codes. Rooms expire after 2 hours automatically.
No user accounts	No login, no registration, no personal data collected or stored.
Transport encryption	WebRTC DataChannels are encrypted with DTLS-SRTP by default.
SocketIO CORS	cors_allowed_origins='*' for development. Restrict in production.
SECRET_KEY env	Flask session key loaded from environment variable in production.
QR size clamped	box_size parameter is clamped to 4–20 to prevent DoS via large QR.
Room cleanup	Background thread deletes rooms older than 2 hours every 5 minutes.
Pending req cleanup	Pending join requests expire after 15 minutes automatically.

Bug Fixes & Changelog

Backend Fixes (B1–B4)

Feature	Description
B1 — Critical	eventlet.monkey_patch() moved to absolute first line before all stdlib imports
B2 — TypeError	leave_room() called with no sid argument (correct API usage)
B3 — qrcode	fill_color / back_color changed to RGB tuples — works all versions
B4 — DoS	QR box_size parameter clamped to range 4–20

Frontend Fixes (J1–J15)

Feature	Description
J1/J9	transfer_stopped socket event listener added — receiver UI no longer hangs
J2	Receiver count uses peerNum (post-increment) not Object.keys().length+1
J3	Division-by-zero guard: peer.total set to max(total, 1)
J4	bufferedAmountLowThreshold set ONCE at DC creation, not inside send loop
J5	rPC.close() called in goHome() before setting rPC=null
J6	closeAllPeers() iterates peers map and calls pc.close() on each
J7	dc.readyState === 'open' checked before every dc.send()
J8	safeSend() wrapper with try-catch around all DataChannel sends
J9	See J1 above
J10	Deep-link bypass: showPage('pgRecv') directly, no pgSend flash
J11	Paste event + setTimeout ensures OTP sync on mobile clipboard paste
J12	Speed meter guards dt<=0 to prevent NaN display
J13	finalizeDownload() only called on all_done, not on individual file_done
J14	goHome() calls closeAllPeers() + rPC.close() before navigation
J15	beforeunload event listener closes all peers before tab closes

Contact & Licensing

Project Licensing

ShareAnywhere is a proprietary project developed by Mr. D.K. Chaudhary. The source code, architecture, UI design, and documentation are the intellectual property of the developer. Unauthorized distribution, resale, or commercial use without written permission is strictly prohibited.

© This project was custom-built for educational and professional use. All rights reserved. © 2025 Mr. D.K. Chaudhary.

Get This Project

If you are interested in this project — for personal use, institutional deployment, or customization — please reach out directly to the developer. Custom features, white-labeling, and deployment support are available on request.

Developer	Handle	Website
Mr. D.K. Chaudhary	@THEDARKGEEKDC	thedarkgeekdc2.github.io

■ Want This Project?

If you want to deploy ShareAnywhere for your school, organization, or personal use, or need custom features added — contact the developer directly.

Kindly Contact:

THEDARKGEEKDC

■ <https://thedarkgeekdc2.github.io>

Primary Teacher & Cyber Security Enthusiast